

Informe Técnico: Sistema de Detección y Resolución de Cubo de Rubik

1. Resumen General

El sistema es una aplicación web interactiva que permite a un usuario cargar imágenes de las caras de un Cubo de Rubik para **detectar sus colores mediante procesamiento de imágenes básico en un Canvas** y, posteriormente, intentar calcular una secuencia de movimientos para "resolverlo" (o al menos ordenar una de sus caras) utilizando un algoritmo de búsqueda en amplitud (**BFS - Breadth-First Search**).

2. Explicación de los Componentes Clave

La aplicación se divide en tres fases principales que funcionan de la siguiente manera:

A. Carga y Procesamiento de Imágenes (Escaneo)

- El usuario sube las fotos de las caras del cubo mediante un `<input type="file" multiple>`.
- Por cada imagen, el código dibuja un lienzo (`<canvas>`) de 300×300 píxeles y calcula el centro de cada una de las 9 sub-casillas (matriz de 3×3).
- Usando el método `ctx.getImageData()`, extrae el color **RGB** exacto del píxel central de cada casilla.

B. Mapeo y Detección de Colores

- El script lee un archivo externo llamado `colores.json` que contiene los valores de referencia para cada color (Blanco, Rojo, Azul, etc.).
- Para saber qué color tiene una casilla, calcula la **distancia euclidiana en el espacio 3D de RGB** mediante la fórmula:

$$Distancia = \sqrt{(R1 - R2)^2 + (G1 - G2)^2 + (B1 - B2)^2}$$

- El color con la menor distancia matemática es el que se le asigna a la casilla y se dibuja en la cuadrícula del navegador.

C. Lógica del Simulador de Movimientos

- El código define 4 movimientos básicos (R1, L1, U1, D1).

D. Algoritmo de Resolución (BFS)

- Al presionar "Resolver Cubo", se dispara una **Búsqueda en Amplitud (BFS)**.

- El algoritmo almacena el estado actual del cubo en una cola y empieza a probar todas las combinaciones posibles de movimientos nivel por nivel.
- Tiene un validador llamado `caraOrdenada()` que verifica si todas las casillas de una misma cara tienen el mismo color que su centro. En cuanto encuentra un camino que logre esto, se detiene y retorna los pasos.
- **Límite de seguridad:** Para evitar que el navegador explote o se quede sin memoria (ya que el espacio de estados de un cubo es gigantesco), el BFS tiene un límite estricto de **4 movimientos máximos**. Si supera eso, se rinde y devuelve una solución genérica quemada en el código (`["R1", "U1", "L1"]`).

E. Animación de los Resultados

- Una vez obtenida la lista de pasos, la función `animarPasos()` utiliza un `setInterval()` para ir mostrando los movimientos en la interfaz de usuario uno por uno cada 700 milisegundos, simulando la ejecución en tiempo real.